



UNIVERSIDAD CATÓLICA ANDRÉS BELLO

Programación Orientada a Objetos

Gabriela Cantos, Steizy Fornica y Amelie Moreno

Maze Hunter - fase 1

1. Pasos que seguimos para la instalación de las librerías:

A través de <https://mvnrepository.com/> descargamos el archivo JAR de la librería que se quería importar.

En IntelliJ:

- En la barra superior en File -> Project Structure
- En la ventana de Project Structure -> Project Settings -> Libraries
- En la parte superior se le da click al icono (+) (New Project Librarie) -> Java -> Seleccionamos el archivo JAR descargado, de la librería a importar.
- Dar click al botón Apply
- Dar click a OK

2. Librerías usadas dentro de cada clase.

AdministradorUsuario:

- `java.util.ArrayList`

Función:

- Implementa una lista dinámica mediante un array redimensionable.

En el código:

- Atributo `private ArrayList<Usuario> usuarios;` almacena los usuarios dentro del juego.
- En el método `registrar()`: crea un array vacío de Estadísticas, para crear un usuario.

- `javax.mail.internet.InternetAddress;`

Función:

- Representa una dirección de email y valida su formato

En el código:

- En el método `validarCorreo()`: se usa para crear la dirección de correo del usuario, y así mismo validarlo

- javax.crypto.Cipher;

Función:

- Proporciona funcionalidades de cifrado y descifrado usando algoritmos criptográficos como AES (nuestro caso)

En el código:

- En el método cifrar(): cifra los datos sensibles del usuario (correo/contraseña) usando AES.

```
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
cipher.init(Cipher.ENCRYPT_MODE, claveSecreta);
byte[] textoCifrado = cipher.doFinal(texto.getBytes("UTF-8"));
```

- En el método descifrar(): descifra los datos almacenados para poder usarlos dentro del código

```
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
cipher.init(Cipher.DECRYPT_MODE, claveSecreta);
byte[] textoPlano =
cipher.doFinal(Base64.getDecoder().decode(textoCifrado));
```

- En el método generarClaveFija(): genera la clave AES que usa Cipher para el cifrado/descifrado

```
byte[] claveBytes = CLAVE_FIJA_BASE64.getBytes("UTF-8");
return new SecretKeySpec(claveBytes, "AES");
```

- Cipher.getInstance("AES/ECB/PKCS5Padding") - Configurar cifrado AES.
- cipher.init(Cipher.ENCRYPT_MODE, claveSecreta) - Inicializar modo cifrado.
- cipher.init(Cipher.DECRYPT_MODE, claveSecreta) - Inicializar modo descifrado.
- cipher.doFinal() - Ejecutar cifrado/descifrado.
- SecretKeySpec - Crear clave AES desde bytes.

- javax.crypto.SecretKey;

Función:

- Es una interfaz que representa una clave criptográfica secreta para algoritmos de cifrado como AES. Proporciona las funcionalidades básicas que toda clave secreta debe implementar.

En el código:

- En el método generarClaveFija(): crea y retorna un objeto SecretKey usando SecretKeySpec con la clave fija en bytes para AES

```
return new SecretKeySpec(claveBytes, "AES");
```

- En el constructor AdministradorUsuario(): inicializa el campo claveSecreta llamando a generarClaveFija()

```
this.claveSecreta = generarClaveFija();
```

- En los métodos cifrar() y descifrar(): utilizan la claveSecreta para configurar el modo de operar del Cipher

```
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
cipher.init(Cipher.ENCRYPT_MODE, claveSecreta);
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
cipher.init(Cipher.DECRYPT_MODE, claveSecreta);
```

- javax.crypto.spec.SecretKeySpec;

Función:

- Es una clase que implementa la interfaz SecretKey y proporciona una representación concreta de una clave secreta a partir de un array de bytes. Se usa para construir claves cuando ya se tiene el material de la clave en forma de bytes.

En el código:

- En el método generarClaveFija(): convierte la cadena CLAVE_FIJA_BASE64 en bytes y crea un SecretKeySpec para AES

```
return new SecretKeySpec(claveBytes, "AES");
```

- java.util.Base64;

Función:

- Proporciona métodos para codificar y decodificar datos en formato Base64, que es un esquema de codificación que convierte datos binarios en texto ASCII

En el código:

- En el método cifrar(): codifica el texto cifrado (bytes binarios) a Base64 para almacenamiento

```
return Base64.getEncoder().encodeToString(textoCifrado);
```

- En el método descifrar(): decodifica el texto Base64 de vuelta a bytes binarios antes del descifrado

```
cipher.doFinal(Base64.getDecoder().decode(textoCifrado));
```

Celda:

- `import com.fasterxml.jackson.annotation.JsonSubTypes;`

Función:

- Sirve para definir los subtipos concretos que JACKSON debe reconocer cuando trabaja con una clase base abstracta o interfaz durante la serialización/deserialización JSON

En el código:

- Anotaciones: ya que celda es una clase abstracta no se pueden crear instancias directas, sin `@JsonSubTypes`, Jackson no sabría qué subclase en concreto debe instanciar al leer el JSON.

```
@JsonSubTypes({
    @JsonSubTypes.Type(value = CaminoLibre.class, name = "camino"),
    @JsonSubTypes.Type(value = Muro.class, name = "pared"),
    @JsonSubTypes.Type(value = Trampa.class, name = "trampa"),
    @JsonSubTypes.Type(value = Cristal.class, name = "cristal"),
    @JsonSubTypes.Type(value = Inicio.class, name = "inicio"),
    @JsonSubTypes.Type(value = Llave.class, name = "llave"),
    @JsonSubTypes.Type(value = VidaExtra.class, name = "vidaextra"),
    @JsonSubTypes.Type(value = Meta.class, name = "meta")
})
```

- `import com.fasterxml.jackson.annotation.JsonTypeInfo;`

Función:

- Sirve para configurar cómo Jackson incluye y maneja la información sobre el tipo real de los objetos durante la serialización/deserialización de jerarquías de clases

En el código:

- Anotación en la clase Celda ya que proporciona el mecanismo que Jackson usará para determinar el tipo concreto durante la deserialización.
- Trabaja con `@JsonSubTypes`. `JsonTypeInfo` define el cómo y `JsonSubTypes` define el qué

```
@JsonTypeInfo(
    use = JsonTypeInfo.Id.NAME,
    include = JsonTypeInfo.As.PROPERTY,
```

```
        property = "tipo"  
    )
```

Estadística:

- import java.time.Duration;

Función:

- Sirve para representar y manipular intervalos de tiempo con precisión.

En el código:

- En el atributo de la clase private Duration tiempoJugado que almacena el tiempo total que duró la partida
- En el constructor recibiendo y asignando el tiempo como atributo

```
public Estadistica(Instant tiempoInicial, Instant tiempoFinal, int  
tamanoDeMatriz,  
                    int cristalesRecolectados, int puntosDeVida, int  
trampasActivadas,  
                    Duration tiempoJugado) {  
    this.tiempoJugado = tiempoJugado;  
}
```

- En el método mostrarEstadistica(): convierte la duracion a un formato legible para mostrarse

```
if (tiempoJugado != null) {  
    long horas = tiempoJugado.toHours();  
    long minutos = tiempoJugado.toMinutes() % 60;  
    long segundos = tiempoJugado.getSeconds() % 60;  
    mostrarTiempo = String.format("%02d:%02d:%02d", horas, minutos,  
segundos);  
}
```

- Si no hay tiempoJugado, calcula la duración entre tiempoInicio y tiempoFinal

```
} else if (tiempoInicio != null && tiempoFinal != null) {  
    Duration duracion = Duration.between(tiempoInicio, tiempoFinal);  
    // ... formateo del tiempo  
}
```

- `import java.time.Instant;`

Función:

- Sirve para representar un punto específico e inmutable de tiempo.

En el código:

- Captura y almacena momentos exactos en el tiempo para el inicio y el fin de la partida
- Almacena en los atributos: `private Instant tiempoInicio; private Instant tiempoFinal;`
- En el constructor para inicializar

```
public Estadistica(Instant tiempoInicial, Instant tiempoFinal, int
    tamanoDeMatriz,
    int cristalesRecolectados, int puntosDeVida, int
    trampasActivadas,
    Duration tiempoJugado) {
    this.tiempoInicio = tiempoInicial;
    this.tiempoFinal = tiempoFinal;
    // ...
}
```

GestorJSON:

- `import com.fasterxml.jackson.databind.ObjectMapper;`

Función:

- Convierte objetos JAVA a JSON. Maneja la serialización/deserialización

En el código:

- Crea el principal motor de la conversión JSON en el constructor `this.objectMapper = new ObjectMapper();`
- En el método `cargarTodosLosUsuarios();` lee el archivo JSON `return objectMapper.readValue(archivo, objectMapper.getTypeFactory().constructCollectionType(List.class, Usuario.class));`
- En el método `guardarUsuarios();` escribe en el archivo JSON `objectMapper.writeValue(new File(ARCHIVO_JSON), usuarios);`

- `import com.fasterxml.jackson.databind.SerializationFeature;`

Función:

- Configura cómo el JACKSON convierte objetos Java - JSON
-

En el código:

- Se emplea en el constructor para: formatear el JSON con indentación, para mayor legibilidad, convertir fechas a formas legibles, evitar errores con clases vacías

```
this.objectMapper.enable(SerializationFeature.INDENT_OUTPUT);
this.objectMapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
this.objectMapper.configure(SerializationFeature.FAIL_ON_EMPTY_BEANS, false);
```

- import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;

Función:

- Agrega soporte para Instant, Duration

En el código:

- Se usa en el constructor para registrar el módulo en ObjectMapper. Permite serializar/deserializar correctamente los elementos Instant y Duration de la clase Estadística

```
this.objectMapper.registerModule(new JavaTimeModule());
```

- import com.fasterxml.jackson.annotation.JsonTypeInfo;

Función:

- Define cómo incluir información de tipo para la serialización polimórfica

En el código:

- En el constructor configura cómo guardar la información, para que Jackson maneje correctamente la herencia al reconocer las subclases de Celda, teniendo en cuenta que un laberinto contiene diferentes tipos

```
this.objectMapper.activateDefaultTyping(
    objectMapper.getPolymorphicTypeValidator(),
    ObjectMapper.DefaultTyping.NON_FINAL,
    JsonTypeInfo.As.PROPERTY
);
```

- import com.fasterxml.jackson.databind.DeserializationFeature;

Función:

- Configura cómo Jackson deserializa, es decir, cómo convierte de JSON a objetos JAVA

En el código:

- En el constructor configura para ignorar propiedades desconocidas, para que al añadir nuevos campos, la información ya cargada que no los poseyera no lanzara errores

```
this.objectMapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
```

- import java.io.IOException;

Función:

- Es una excepción que representa errores de entrada/salida durante las operaciones que se manejan dentro del juego.

En el código:

- Se usa en los siguientes métodos, donde se maneja el archivo JSON

```
public List<Usuario> cargarTodosLosUsuarios() throws IOException {
public void guardarUsuarios(List<Usuario> usuarios) throws
IOException {
public void guardarEstadoCompleto(Usuario usuarioActual) throws
IOException {
public Usuario cargarPartidaUsuario(String correo) throws
IOException {
public void guardarEstadistica(Usuario usuario, Estadistica
nuevaEstadistica) throws IOException {
```

- En los métodos crearArchivoJSON() y mostrarArchivoJSON() en caso de errores con el manejo del archivo

*cargarArchivoJSON

```
try {
    File archivo = new File(ARCHIVO_JSON);
    if (!archivo.exists()) {
        archivo.createNewFile();
        objectMapper.writeValue(archivo, new ArrayList<Usuario>());
    }
} catch (IOException e) {
    System.out.println("Error al crear archivo JSON: " + e.getMessage());
}
```

*mostrarArchivoJSON()

```
try {
    List<Usuario> usuarios = cargarTodosLosUsuarios();
} catch (IOException e) {
    System.out.println("Error al leer archivo JSON: " + e.getMessage());
}
```

- `import java.io.File;`

Función:

- Representa archivos y directorios del sistema

En el código:

- En el método `crearArchivoJson()`: verifica y crea el archivo JSON

```
File archivo = new File(ARCHIVO_JSON);
if (!archivo.exists()) {
    archivo.createNewFile();
}
```

- En el método `cargarTodosLosUuarios()`: verifica que exista el archivo JSON

```
File archivo = new File(ARCHIVO_JSON);
if (!archivo.exists()) {
    return new ArrayList<>();
}
```

- En el método `guardarUsuarios()`: crea un objeto `File` que representa la ruta física y concreta en donde se guardará el archivo JSON.

```
objectMapper.writeValue(new File(ARCHIVO_JSON), usuarios);
```

- `import java.util.ArrayList;`

Función:

- Es una implementación de lista dinámica que usa un array redimensionable

En el código:

- En el método `crearArchivoJson()`: para crear el archivo JSON con una estructura válida, es decir, con un array vacío
`objectMapper.writeValue(archivo, new ArrayList<Usuario>());`
- En el método `cargarTodosLosUsuarios()`: para retornar una lista nueva en caso que el archivo no exista

```
if (!archivo.exists()) {
    return new ArrayList<>();
}
```
- En el método `guardarEstadoCompleto()`: crea una nueva instancia para guardar en la lista de administrador usuarios
`administradorUsuario.setUsuarios(new ArrayList<>(todosLosUsuarios));`
- En el método `guardarEstadistica()`: inicializa la lista

```

        if (usuario.getEstadisticas() == null) {
            usuario.setEstadisticas(new ArrayList<>());
        }
    }

```

- `import java.util.List;`

Función:

- Es una interfaz que define un molde para hacer listas, son reglas que todas las listas deben seguir

En el código:

- Se usa para recibir listas de diferente tipo de usuarios, en los métodos `cargarTodosLosUsuarios` y `guardarUSuarios`

Laberinto:

- `import java.util.ArrayList;`

Función:

- Es una implementación de lista dinámica que usa un array redimensionable

En el código:

- En el método `generarLaberinto()`: para crear una nueva lista vacía y para agregar elementos a ella. Es una lista que almacena las direcciones

```

List<int[]> dirList = new ArrayList<>(); // Constructor
for (int[] dir : direcciones) {
    dirList.add(dir); // Método add()
}

```

- `import java.util.Collections;`

Función:

- Es una clase que agrupa métodos estáticos para operar sobre colecciones

En el código:

- En `generarLaberinto()`: se usa para mezclar aleatoriamente la lista de direcciones
`Collections.shuffle(dirList, random);`

- `import java.util.List;`

Función:

- Define las operaciones básicas de una lista ordenada

En el código:

- En `generarLaberinto()`: para recorrer la lista mezclada. Define el tipo de variable `dirList`

```
List<int[]> dirList = new ArrayList<>();
for (int[] dir : dirList) {
}
```

- import java.util.Random;

Función:

- Genera números aleatorios

En el código:

- En el constructor para crear la instancia del generador
- En colocarElemento(): genera un numero random para la posicion del elemento dentro del laberinto
- En crearLaberinto(): genera posiciones aleatorias del inicio y meta. Asegura que cada laberinto sea único
Se pasa como parámetro en Collections.shuffle()

```
private Random random;
this.random = new Random();
```

```
int x = random.nextInt(tamano - 2) + 1;
int inicioX = random.nextInt((tamano - 2) / 2) * 2 + 1;
int inicioY = random.nextInt((tamano - 2) / 2) * 2 + 1;
int metaX = random.nextInt((tamano - 2) / 2) * 2 + 1;
int metaY = random.nextInt((tamano - 2) / 2) * 2 + 1;
```

Main:

- import java.io.IOException;

Función:

- Maneja excepciones de entrada/salida durante las operaciones

En el código:

- En guardarPartida()


```
try {
    gestorJSON.guardarEstadoCompleto(usuarioActual);
} catch (IOException e) {
    System.out.println(ROJO + ">> Error al guardar partida: " +
e.getMessage() + RESET);
}
```
- En finalizarPartida()


```
try {
    gestorJSON.guardarEstadistica(usuarioActual, estadistica);
} catch (IOException e) {
```

```

        System.out.println(ROJO + ">> Error al guardar estadística: " +
e.getMessage() + RESET);
    }

```

- En mostrarMenuJuego() en la opción 2 al cargar la partida guardada

```

try {

```

```

    Usuario usuarioCargado =
gestorJSON.cargarPartidaUsuario(correo);
} catch (IOException e) {
    System.out.println(ROJO + ">> Error al cargar partida: " +
e.getMessage() + RESET);
}

```

- En el main() en el registro del usuario

```

try {

```

```

    gestorJSON.guardarEstadoCompleto(nuevoUsuario);
} catch (IOException e) {
    System.out.println(ROJO + ">> Error al guardar usuario: " +
e.getMessage() + RESET);
}

```

- Se usa ya que todas las operaciones de guardar o cargar con
GestorJSON pueden fallar por problemas de archivo

- import java.time.Duration;

Función:

- Representa y manipula intervalos de tiempo

En el código:

- En formatTiempo(): convierte Duration a un formato hh:mm:ss

```

private static String formatTiempo(Duration duracion) {
    long horas = duracion.toHours();
    long minutos = duracion.toMinutes() % 60;
    long segundos = duracion.getSeconds() % 60;
    return String.format("%02d:%02d:%02d", horas, minutos, segundos);
}

```

- En jugarPartida(): muestra el tiempo transcurrido

```

Duration tiempoTranscurrido = partida.obtenerTiempoTranscurrido();
System.out.println(AZUL_CLARO + "🕒 Tiempo: " + AMARILLO +
formatTiempo(tiempoTranscurrido) + RESET);

```
- finalizarPartida(): obtiene el tiempo total jugado

```

Duration tiempoJugado = partida.finalizarPartida();

```

- En `mostrarMenuJuego()`: obtiene el tiempo jugado desde las estadísticas en la opción 3 del menú

```
if (estadistica.getTiempoJugado() != null) {
    Duration tiempo = estadistica.getTiempoJugado();
    // ... formateo
}
```

- `import java.util.ArrayList;`

Función:

- Sirve para implementar listas dinámicas

En el código:

- Lo usa en el main para inicializar la lista de los usuarios

```
ArrayList<Usuario> usuarios = new ArrayList<>();
usuarios = new ArrayList<>(gestorJSON.cargarTodosLosUsuarios());
```
- En `mostrarMenuJuego()`: en la opción 3, guarda en una lista las estadísticas que obtiene del usuario actual y la recorre para mostrarlas

```
ArrayList<Estadistica> estadisticas = usuarioActual.getEstadisticas();
for (int i = 0; i < estadisticas.size(); i++) {
    Estadistica estadistica = estadisticas.get(i);
    // ... mostrar estadísticas
}
```

- `import java.util.Scanner;`

Función:

- Lee la entrada del usuario desde la consola

En el código:

- Declara globalmente el scanner

```
private static Scanner scanner = new Scanner(System.in);
```
- En `jugarPartida()`: hace la lectura de los movimientos ingresados por el usuario

```
String movimiento = scanner.nextLine().trim().toLowerCase();
String confirmacion = scanner.nextLine().trim().toLowerCase();
```
- En `leerEntero()`: lee un número ingresado por el usuario

```
private static int leerEntero() {
    while (true) {
```

```

        try {
            return Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.print(ROJO + ">> Ingrese un número válido: " +
RESET);
        }
    }
}

```

- En mostrarMenuJuego(): para leer la opcion ingresada por el usuario
- En main(): para el registro y el login del usuario

```

int opcion = leerEntero();
String respuesta = scanner.nextLine().trim().toLowerCase();
String correo = scanner.nextLine().trim();
String contrasenia = scanner.nextLine();
String confirmarContrasenia = scanner.nextLine();

```

Partida:

- import java.time.Instant;

Función:

- Representa un punto específico en el tiempo

En el código:

- Declara los atributos de la clase: private Instant tiempoInicio; private Instant tiempoFinal;
- En iniciarPartida(): establece el tiempo de inicio
this.tiempoInicio = Instant.now();
- En pausarTiempo(): usa instant.now para que se pueda calcular la duración entre el tiempo de inicio y el tiempo actual
Duration tiempoTranscurrido = Duration.between(tiempoInicio, Instant.now());
- En reanudarTiempo(): reestablece el timestamp del tiempo de inicio
tiempoInicio = Instant.now();
- En obtenerTiempoTranscurrido(): calcula la duracion del tiempo hasta el momento
tiempoTotal.plus(Duration.between(tiempoInicio, Instant.now()));
- En finalizarPartida(): registra el timestamp final
this.tiempoFinal = Instant.now();

- import java.time.Duration;

Función:

- Representa y manipula intervalos de tiempo

En el código:

- Declara el tiempo total acumulado de la partida private Duration tiempoAcumulado;
- Se utiliza en el constructor para inicializar el tiempo acumulado this.tiempoAcumulado = Duration.ZERO;
- En pausarTiempo(): para calcular y acumular la duración
Duration tiempoTranscurrido = Duration.between(tiempoInicio, Instant.now());
tiempoAcumulado = tiempoAcumulado.plus(tiempoTranscurrido);
- En obtenerTiempoTranscurrido(): retorna la duración total del tiempo
Duration tiempoTotal = tiempoAcumulado;
if (tiempoInicio != null) {
 tiempoTotal = tiempoTotal.plus(Duration.between(tiempoInicio, Instant.now()));
}
return tiempoTotal;
- En finalizarPartida(): calcula la duración final
Duration tiempoTranscurrido = Duration.between(tiempoInicio, Instant.now());
tiempoAcumulado = tiempoAcumulado.plus(tiempoTranscurrido);
return tiempoAcumulado;

Usuario:

- import java.util.ArrayList

Función:

- Implementa una lista dinámica que permite almacenar elementos en orden con accesor por índice y crecimiento automático

En el código:

- Declara el atributo de estadísticas como una lista para almacenarlas private ArrayList<Estadistica> estadisticas;
- En el constructor para inicializar la lista
public Usuario(String correo, String contrasenia, Partida partida, ArrayList<Estadistica> estadisticas) {
 this.estadisticas = estadisticas;

```
}
```

- En los getters y setters para guardar y retornar la lista de estadísticas

```
public ArrayList<Estadistica> getEstadisticas() {
```

```
    return estadisticas;
```

```
}
```

```
public void setEstadisticas(ArrayList<Estadistica> estadisticas) {
```

```
    this.estadisticas = estadisticas;
```

```
}
```